STM32L476RG ADC Input on PA0

Bare-metal register setup, 12-bit conversion, and screen display

Target board	STM32L476RG / NUCLEO-L476RG
Analog input	PA0, Arduino header A0, ADC1 channel 5
Input hardware	10 kΩ potentiometer wired between 3.3 V and GND
Display output	Raw ADC value, voltage in volts, and a horizontal bar graph

Before starting

- Use a board powered from USB or a regulated 3.3 V supply.
- Keep the PA0 input voltage between 0 V and 3.3 V.
- Have the screen driver already working before adding the ADC files.
- Use short jumper wires and a common ground between the potentiometer and the board.

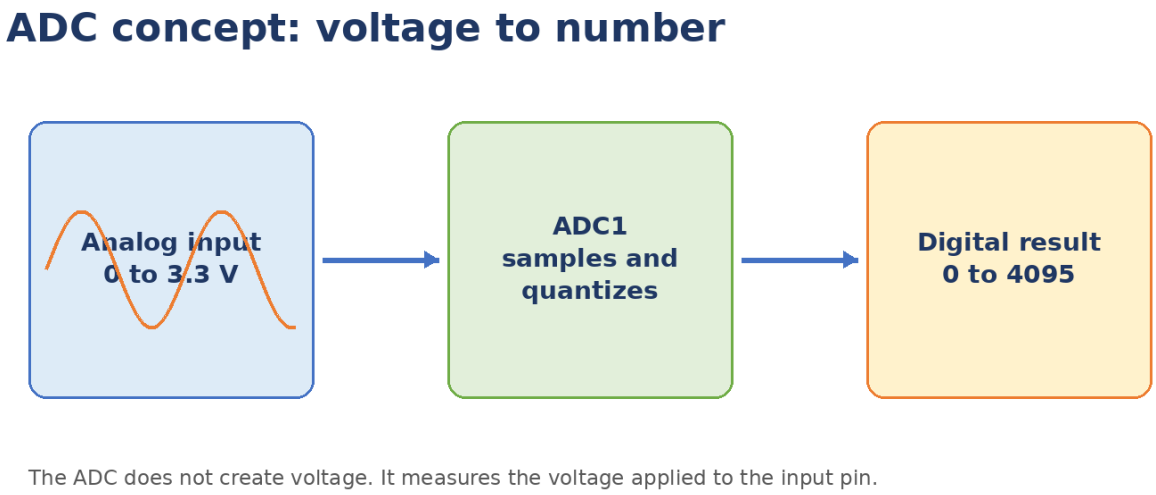

Figure 1. ADC concept: an analog voltage is sampled and converted to a digital number.

1. Purpose of the guide

This guide shows how to read an analog voltage on PA0 using ADC1 on the STM32L476RG. The value is read with bare-metal register access, converted to millivolts, and shown on the existing display.

The program uses a single ADC channel in polling mode. The CPU starts one conversion, waits for the end-of-conversion flag, reads ADC1->DR, and repeats inside the main loop.

What this guide covers

- Wiring a potentiometer as a safe 0 V to 3.3 V input source.
- Configuring PA0 as an analog input and connecting its analog switch.
- Enabling ADC1, its regulator, calibration, and ready flag.
- Selecting channel 5 as the only regular conversion in the sequence.
- Choosing a practical ADC clock and sample time for a slow potentiometer input.
- Displaying the raw value, voltage, and a bar graph on the screen.

2. Parts list and preparation

Component	Qty	Notes
STM32L476RG board	1	NUCLEO-L476RG recommended
10 kΩ potentiometer	1	Used as an adjustable voltage divider
Breadboard	1	Standard solderless breadboard
Jumper wires	3+	3.3 V, PA0, and GND
USB cable	1	Programming and board power
Safety rule: Do not connect 5 V to PA0. On this board, the ADC input must remain between GND and the 3.3 V analog supply range.		

3. ADC theory: voltage to number

ADC means Analog-to-Digital Converter. The ADC does not create voltage. It measures the voltage already present on the input pin and returns a digital number.

With 12-bit resolution, the result range is 0 to 4095. A value near 0 means PA0 is near 0 V. A value near 4095 means PA0 is near the reference voltage. With a 3.3 V reference, mid-scale is about 1.65 V.

```
millivolts = ((uint32_t)adc_value * 3300UL) / 4095UL;
```

PA0 voltage	Expected ADC result	Meaning
0 V	about 0	Bottom of the range
1.65 V	about 2048	Half scale
3.3 V	about 4095	Top of the range
Above 3.3 V	Not allowed	Risk of damaging the pin

Polarity and reference

This ADC input is single-ended. The voltage is measured relative to GND, so a higher voltage on PA0 gives a larger digital result. There is no clock-polarity or idle-state setting like I2C or SPI; the important settings are the ADC clock source, sample time, channel selection, resolution, and voltage reference.

4. Hardware wiring

Wire the potentiometer as a voltage divider. One outside pin goes to 3.3 V, the other outside pin goes to GND, and the middle wiper pin goes to PA0 / A0.

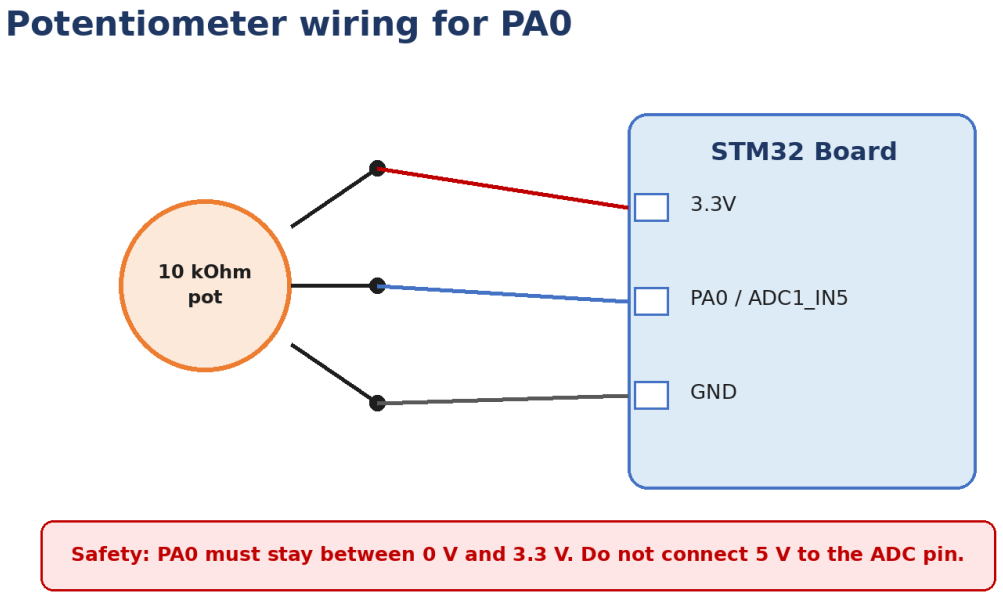

Figure 2. Potentiometer wiring for PA0.

Potentiometer pin	Connect to	Reason
Outside pin 1	3.3 V	Top of the voltage divider
Middle pin / wiper	PA0 / A0	Variable voltage input to ADC1 channel 5
Outside pin 2	GND	Bottom of the voltage divider
Before power-on: Check the wiper with a multimeter. Turning the knob should sweep PA0 from about 0 V to about 3.3 V, not 5 V.		

5. Project files

The screen driver is separate from the ADC code. The ADC files only measure PA0 and return numbers. main.c decides how those numbers are displayed.

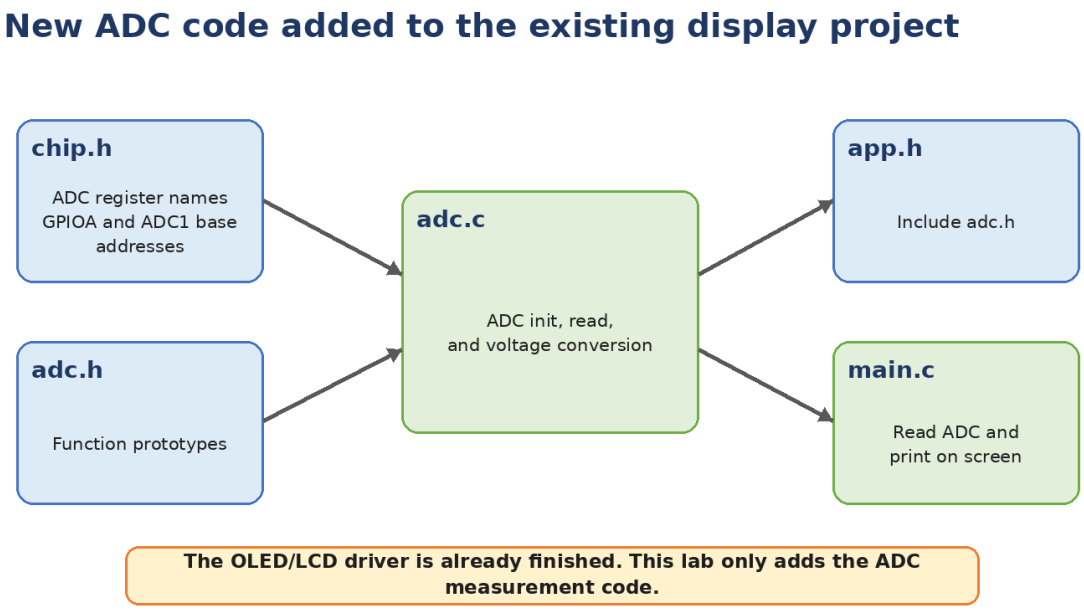

Figure 3. ADC files added to the existing display project.

File	Job in the project
chip.h	Register structures, peripheral base addresses, and bit masks.
adc.h	Function prototypes used by main.c.
adc.c	ADC initialization, single conversion read, and millivolt conversion.
app.h	Common include file used by main.c.
main.c	Reads the ADC value and prints the result on the screen.

6. Register map used in this example

This section connects each C register access to the STM32L476RG memory map. The code uses C structures so the program can write registers with readable names instead of raw addresses.

Register	Address	Field used	Purpose
RCC_AHB2ENR	0x4002104C	GPIOAEN bit 0	Turns on the GPIOA peripheral clock.
RCC_AHB2ENR	0x4002104C	ADCEN bit 13	Turns on the ADC peripheral clock.
GPIOA_MODER	0x48000000	PA0 bits [1:0] = 11	Selects analog mode for PA0.
GPIOA_PUPDR	0x4800000C	PA0 bits [1:0] = 00	Disables pull-up and pull-down resistors.
GPIOA_ASCR	0x4800002C	ASC0 bit 0	Connects PA0 to the analog input path.
ADC_COMMON_CCR	0x50040308	CKMODE bits [17:16] = 01	Uses HCLK as the ADC clock.
ADC1_CR	0x50040008	DEEPPWD, ADVREGEN, ADCAL, ADEN, ADSTART	Leaves deep power-down, enables regulator, calibrates, enables, and starts conversion.
ADC1_ISR	0x50040000	ADRDY, EOC, EOS	Ready and conversion-complete flags.
ADC1_CFGR	0x5004000C	reset value used	Single conversion, right-aligned, 12-bit default.
ADC1_SMPR1	0x50040014	SMP5 bits = 100	Sets channel 5 sample time to 47.5 ADC cycles.
ADC1_SQR1	0x50040030	SQ1 = 5, L = 0	Selects one regular conversion: channel 5.
ADC1_DR	0x50040040	DATA bits [15:0]	Holds the conversion result after EOC is set.

Why PA0 is channel 5

On the NUCLEO-L476RG board, the Arduino analog pin A0 is connected to STM32 pin PA0. On this device, PA0 is available as ADC1/ADC2 input channel 5. That is why the code uses ADC_CHANNEL_PA0 = 5U and places channel 5 into SQR1.

ADC timing used here

Setting	Code value	Reason
ADC clock source	CKMODE = 01	Uses HCLK. With the default MSI clock used here, this gives a simple low-speed ADC clock.
Sample time	SMP5 = 100	47.5 ADC cycles. This is longer than the minimum and works well with a potentiometer and jumper wires.
12-bit conversion time	12.5 ADC cycles	Default 12-bit conversion adds 12.5 cycles after sampling.
Approximate total	47.5 + 12.5 = 60 cycles	At 4 MHz ADC clock, one conversion takes about 15 microseconds. The screen update is much slower, so the ADC is not the bottleneck.

A longer sample time gives the internal sampling capacitor more time to settle. That is useful when the input comes from a potentiometer, long jumper wires, or a source that cannot charge the ADC input instantly.

7. Add ADC definitions to chip.h

Place the ADC register structures and bit masks before the final #endif in chip.h. These definitions match the register accesses used by adc.c.

```
/* Add these ADC definitions to chip.h */
typedef struct {
    __IO uint32_t ISR;
    __IO uint32_t IER;
    __IO uint32_t CR;
    __IO uint32_t CFGR;
    __IO uint32_t CFGR2;
    __IO uint32_t SMPR1;
    __IO uint32_t SMPR2;
    uint32_t RESERVED0;
    __IO uint32_t TR1;
    __IO uint32_t TR2;
    __IO uint32_t TR3;
    uint32_t RESERVED1;
    __IO uint32_t SQR1;
    __IO uint32_t SQR2;
    __IO uint32_t SQR3;
    __IO uint32_t SQR4;
    __IO uint32_t DR;
} adc_regs_t;

typedef struct {
    __IO uint32_t CSR;
    uint32_t RESERVED;
    __IO uint32_t CCR;
    __IO uint32_t CDR;
} adc_common_regs_t;

#define GPIOA      ((gpio_regs_t *)0x48000000UL)
#define ADC1       ((adc_regs_t *)0x50040000UL)
#define ADC_COMMON ((adc_common_regs_t *)0x50040300UL)

#define RCC_AHB2ENR_GPIOAEN (1UL << 0)
#define RCC_AHB2ENR_ADCEN   (1UL << 13)

#define ADC_ISR_ADRDY (1UL << 0)
#define ADC_ISR_EOC   (1UL << 2)
#define ADC_ISR_EOS   (1UL << 3)

#define ADC_CR_ADEN     (1UL << 0)
#define ADC_CR_ADDIS    (1UL << 1)
#define ADC_CR_ADSTART  (1UL << 2)
#define ADC_CR_ADVREGEN (1UL << 28)
#define ADC_CR_DEEPPWD  (1UL << 29)
#define ADC_CR_ADCAL    (1UL << 31)

#define ADC_CCR_CKMODE_0 (1UL << 16)
```

8. Create adc.h

adc.h gives main.c a small, clear interface. main.c does not need to know which ADC registers are touched.

```
#ifndef ADC_H
#define ADC_H

#include <stdint.h>

void adc1_pa0_init(void);
uint16_t adc1_read(void);
uint32_t adc_to_millivolts(uint16_t adc_value);

#endif
```

9. Create adc.c

adc.c contains the hardware setup. The order matters: clocks first, PA0 analog configuration, ADC regulator, calibration, channel selection, ADC enable, then conversions.

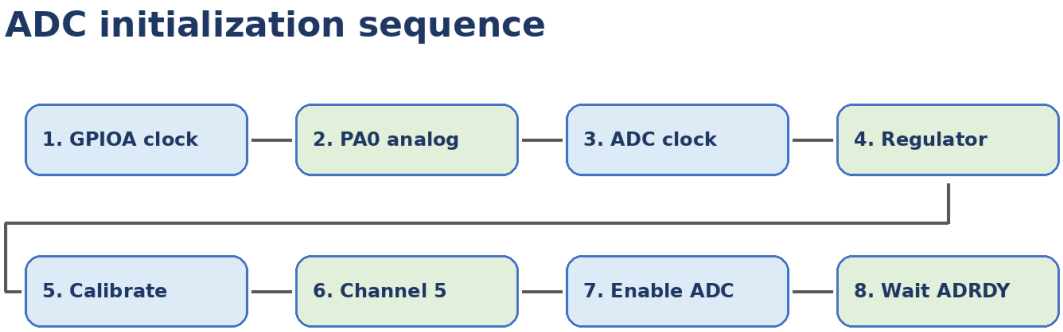

Figure 4. ADC initialization sequence.

```
#include "chip.h"
#include "adc.h"

#define ADC_CHANNEL_PA0 5U
#define ADC_MAX_VALUE   4095UL
#define VREF_MV         3300UL

static void adc_delay(void)
{
    for (volatile uint32_t i = 0; i < 10000UL; i++) {
    }
}

void adc1_pa0_init(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER |= (3UL << (0U * 2U));
    GPIOA->PUPDR &= ~(3UL << (0U * 2U));
    GPIOA->ASCR  |= (1UL << 0U);

    RCC->AHB2ENR |= RCC_AHB2ENR_ADCEN;

    ADC_COMMON->CCR &= ~(3UL << 16U);
    ADC_COMMON->CCR |= ADC_CCR_CKMODE_0;

    ADC1->CR &= ~ADC_CR_DEEPPWD;
    ADC1->CR |= ADC_CR_ADVREGEN;
    adc_delay();

    if ((ADC1->CR & ADC_CR_ADEN) != 0U) {
        ADC1->CR |= ADC_CR_ADDIS;
        while ((ADC1->CR & ADC_CR_ADEN) != 0U) {
        }
    }

    ADC1->CR |= ADC_CR_ADCAL;
    while ((ADC1->CR & ADC_CR_ADCAL) != 0U) {
    }

    ADC1->CFGR = 0x00000000UL;
```

```
    ADC1->SMPR1 &= ~(7UL << (ADC_CHANNEL_PA0 * 3U));
    ADC1->SMPR1 |=  (4UL << (ADC_CHANNEL_PA0 * 3U));


    ADC1->SQR1 = 0x00000000UL;
    ADC1->SQR1 |= (ADC_CHANNEL_PA0 << 6U);


    ADC1->ISR = ADC_ISR_ADRDY;
    ADC1->CR |= ADC_CR_ADEN;
    while ((ADC1->ISR & ADC_ISR_ADRDY) == 0U) {
    }
}
```

```
uint16_t adc1_read(void)
{
    ADC1->ISR = ADC_ISR_EOC | ADC_ISR_EOS;
    ADC1->CR |= ADC_CR_ADSTART;


    while ((ADC1->ISR & ADC_ISR_EOC) == 0U) {
    }


    return (uint16_t)(ADC1->DR & 0x0FFFU);
}


uint32_t adc_to_millivolts(uint16_t adc_value)
{
    return ((uint32_t)adc_value * VREF_MV) / ADC_MAX_VALUE;
}
```

10. Step-by-step ADC initialization

1. Enable GPIOA clock: RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN; Without this clock, writes to GPIOA registers do not configure the pin.

2. Put PA0 in analog mode: GPIOA->MODER uses two bits per pin. For PA0, 11 selects analog mode and disconnects the normal digital input/output path.

3. Disable pulls on PA0: GPIOA->PUPDR is cleared for PA0. A pull-up or pull-down resistor would change the measured voltage from the potentiometer.

4. Connect the analog switch: GPIOA->ASCR sets bit 0. On STM32L4, this connects PA0 to the internal analog path used by the ADC.

5. Enable the ADC clock: RCC->AHB2ENR |= RCC_AHB2ENR_ADCEN; The ADC peripheral must be clocked before its registers can be used reliably.

6. Select the ADC clock mode: ADC_COMMON->CCR sets CKMODE = 01. The ADC clock follows HCLK. This is simple and slow enough for a manually turned potentiometer.

7. Leave deep-power-down and enable the regulator: ADC1->CR clears DEEPPWD and sets ADVREGEN. The internal ADC voltage regulator must be running before calibration and conversion.

8. Calibrate ADC1: ADC1->CR sets ADCAL. The code waits until ADCAL clears, which means calibration is finished.

9. Use basic single-conversion configuration: ADC1->CFGR = 0 keeps the default 12-bit resolution, right alignment, software trigger, and single conversion mode.

10. Set sample time for channel 5: SMPR1 chooses 47.5 ADC clock cycles. This gives the ADC sampling capacitor more time to settle.

11. Select one channel in the regular sequence: SQR1 sets SQ1 = 5 and leaves L = 0, which means one conversion in the sequence: PA0 / channel 5.

12. Enable ADC and wait for ADRDY: ADEN starts the ADC. ADRDY confirms that ADC1 is ready to accept conversion starts.

Reading one sample

adc1_read() clears old EOC and EOS flags, starts a new regular conversion with ADSTART, waits for EOC, and then reads ADC1->DR. The mask 0x0FFF keeps only the 12-bit result.

Why polling mode works here

Polling mode keeps the code easy to follow. The program waits until the conversion is finished before reading the result. This is fine for a slow knob input and a display that updates much slower than the ADC conversion itself.

11. Use main.c to read and display the value

main.c reads the ADC value, converts it to millivolts, formats two text lines, draws the bar graph, and updates the screen buffer.

Example screen output

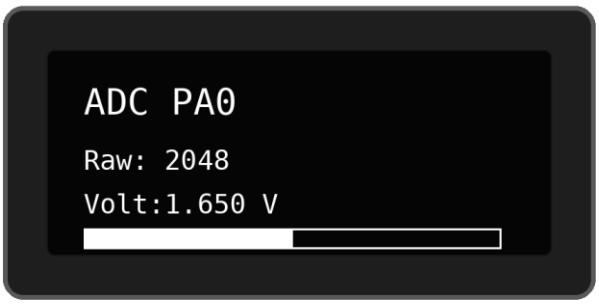

Figure 5. Example screen output for a mid-scale ADC input.

```
#include <stdio.h>
#include "app.h"


static void draw_bar(uint16_t adc_value)
{
    uint32_t bar_width = ((uint32_t)adc_value * OLED_WIDTH) / 4095UL;


    for (uint16_t x = 0U; x < OLED_WIDTH; x++) {
        for (uint16_t y = 54U; y < 63U; y++) {
            oled_pixel(x, y, (x < bar_width) ? OLED_WHITE : OLED_BLACK);
        }
    }
}


int main(void)
{
    uint16_t adc_value;
    uint32_t millivolts;
    char text[24];


    oled_init();
    adc1_pa0_init();


    while (1) {
        adc_value = adc1_read();
        millivolts = adc_to_millivolts(adc_value);


        oled_clear(OLED_BLACK);


        oled_cursor(0, 0);
        oled_text("ADC PA0", &font_11x18, OLED_WHITE);


        oled_cursor(0, 22);
        snprintf(text, sizeof(text), "Raw: %4u", adc_value);
        oled_text(text, &font_7x10, OLED_WHITE);


        oled_cursor(0, 36);
        snprintf(text, sizeof(text), "Volt:%lu.%03lu V",
                 millivolts / 1000UL,
                 millivolts % 1000UL);
        oled_text(text, &font_7x10, OLED_WHITE);


        draw_bar(adc_value);
        oled_update();
    }
}
```

How the bar graph works

bar_width maps the ADC range 0 to 4095 onto the display width. If the ADC value is 2048, the bar is about half of the screen width. The loop draws white pixels before bar_width and black pixels after it.

12. Step-by-step lab procedure

1. Wire the potentiometer: outside pins to 3.3 V and GND, middle pin to PA0 / A0.
2. Measure the wiper voltage before connecting USB power to the code. It must stay between 0 V and 3.3 V.
3. Add the ADC register definitions to chip.h.
4. Create adc.h and adc.c.
5. Include adc.h from app.h.
6. Build the project and flash the board.
7. Turn the potentiometer slowly and watch the raw value, voltage, and bar graph change.
8. Measure PA0 with a multimeter and compare it with the voltage shown on the screen.

13. Troubleshooting

Figure 6. ADC troubleshooting checklist.

Symptom	What to check
Value always 0	PA0 may be tied to GND, the wiper may be on the wrong pin, or GPIOA/ADC clocks may not be enabled.
Value always 4095	PA0 may be tied to 3.3 V, floating high, or the potentiometer outside pins may be swapped or disconnected.
Value jumps too much	Use shorter wires, check common ground, and keep the longer sample time.
Screen does not change	Confirm oled_update() runs after drawing and adc1_read() is called inside the loop.
Program stuck during init	Check ADC clock, ADC regulator delay, calibration loop, and ADRDY flag.
Voltage reading is off	Measure the actual 3.3 V rail. The conversion formula assumes VREF is 3300 mV.

Quick debug checks

- Short PA0 to GND briefly through a jumper: the value should go near 0.
- Connect PA0 to 3.3 V briefly: the value should go near 4095.
- Leave PA0 connected to the potentiometer wiper for normal operation; do not leave it floating.

- Use the debugger to watch adc_value and millivolts while turning the knob.

14. Review questions

9. What is the numeric range of a 12-bit ADC result?
10. Why is PA0 placed in analog mode instead of input mode?
11. Why are the pull-up and pull-down resistors disabled on PA0?
12. What does the EOC flag mean?
13. Why does this example use channel 5 for PA0?
14. What changes in the millivolt formula if the reference voltage is 3.0 V instead of 3.3 V?
15. Why can a longer sample time make the reading more stable?
16. Why is polling mode acceptable for this display demo?

15. Appendix A: register address reference

Name	Address	Notes
RCC_AHB2ENR	0x4002104C	Enables GPIOA and ADC clocks.
GPIOA_MODER	0x48000000	PA0 mode bits.
GPIOA_PUPDR	0x4800000C	PA0 pull configuration.
GPIOA_ASCR	0x4800002C	Analog switch control.
ADC_COMMON_CCR	0x50040308	ADC common clock mode.
ADC1_ISR	0x50040000	ADRDY, EOC, EOS flags.
ADC1_CR	0x50040008	Enable, disable, calibration, start.
ADC1_CFGR	0x5004000C	Resolution, alignment, trigger mode.
ADC1_SMPR1	0x50040014	Sample time for channels 0 to 9.
ADC1_SQR1	0x50040030	Regular sequence length and first channel.
ADC1_DR	0x50040040	Conversion data register.

16. Appendix B: complete code listing

The following listing groups the three pieces used in this example: chip.h additions, adc.h, and adc.c. The main.c display loop is shown earlier in the guide.

chip.h additions

```
/* Add these ADC definitions to chip.h */
typedef struct {
    __IO uint32_t ISR;
    __IO uint32_t IER;
    __IO uint32_t CR;
    __IO uint32_t CFGR;
    __IO uint32_t CFGR2;
    __IO uint32_t SMPR1;
    __IO uint32_t SMPR2;
    uint32_t RESERVED0;
    __IO uint32_t TR1;
    __IO uint32_t TR2;
    __IO uint32_t TR3;
    uint32_t RESERVED1;
    __IO uint32_t SQR1;
    __IO uint32_t SQR2;
    __IO uint32_t SQR3;
    __IO uint32_t SQR4;
    __IO uint32_t DR;
} adc_regs_t;

typedef struct {
    __IO uint32_t CSR;
    uint32_t RESERVED;
    __IO uint32_t CCR;
    __IO uint32_t CDR;
} adc_common_regs_t;

#define GPIOA      ((gpio_regs_t *)0x48000000UL)
#define ADC1       ((adc_regs_t *)0x50040000UL)
#define ADC_COMMON ((adc_common_regs_t *)0x50040300UL)

#define RCC_AHB2ENR_GPIOAEN (1UL << 0)
#define RCC_AHB2ENR_ADCEN   (1UL << 13)

#define ADC_ISR_ADRDY (1UL << 0)
#define ADC_ISR_EOC   (1UL << 2)
#define ADC_ISR_EOS   (1UL << 3)

#define ADC_CR_ADEN     (1UL << 0)
#define ADC_CR_ADDIS    (1UL << 1)
#define ADC_CR_ADSTART  (1UL << 2)
#define ADC_CR_ADVREGEN (1UL << 28)
#define ADC_CR_DEEPPWD  (1UL << 29)
#define ADC_CR_ADCAL    (1UL << 31)

#define ADC_CCR_CKMODE_0 (1UL << 16)
```

adc.h

```
#ifndef ADC_H
#define ADC_H

#include <stdint.h>

void adc1_pa0_init(void);
uint16_t adc1_read(void);
uint32_t adc_to_millivolts(uint16_t adc_value);

#endif
```

adc.c

```
#include "chip.h"
#include "adc.h"


#define ADC_CHANNEL_PA0 5U
#define ADC_MAX_VALUE   4095UL
#define VREF_MV         3300UL


static void adc_delay(void)
{
    for (volatile uint32_t i = 0; i < 10000UL; i++) {
    }
}


void adc1_pa0_init(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;


    GPIOA->MODER |= (3UL << (0U * 2U));
    GPIOA->PUPDR &= ~(3UL << (0U * 2U));
    GPIOA->ASCR  |= (1UL << 0U);


    RCC->AHB2ENR |= RCC_AHB2ENR_ADCEN;


    ADC_COMMON->CCR &= ~(3UL << 16U);
    ADC_COMMON->CCR |= ADC_CCR_CKMODE_0;


    ADC1->CR &= ~ADC_CR_DEEPPWD;
    ADC1->CR |= ADC_CR_ADVREGEN;
    adc_delay();


    if ((ADC1->CR & ADC_CR_ADEN) != 0U) {
        ADC1->CR |= ADC_CR_ADDIS;
        while ((ADC1->CR & ADC_CR_ADEN) != 0U) {
        }
    }


    ADC1->CR |= ADC_CR_ADCAL;
    while ((ADC1->CR & ADC_CR_ADCAL) != 0U) {
    }


    ADC1->CFGR = 0x00000000UL;


    ADC1->SMPR1 &= ~(7UL << (ADC_CHANNEL_PA0 * 3U));
    ADC1->SMPR1 |=  (4UL << (ADC_CHANNEL_PA0 * 3U));


    ADC1->SQR1 = 0x00000000UL;
    ADC1->SQR1 |= (ADC_CHANNEL_PA0 << 6U);


    ADC1->ISR = ADC_ISR_ADRDY;
    ADC1->CR |= ADC_CR_ADEN;
    while ((ADC1->ISR & ADC_ISR_ADRDY) == 0U) {
    }
}


uint16_t adc1_read(void)
{
    ADC1->ISR = ADC_ISR_EOC | ADC_ISR_EOS;
    ADC1->CR |= ADC_CR_ADSTART;


    while ((ADC1->ISR & ADC_ISR_EOC) == 0U) {
    }


    return (uint16_t)(ADC1->DR & 0x0FFFU);
```

```
}

uint32_t adc_to_millivolts(uint16_t adc_value)
{
    return ((uint32_t)adc_value * VREF_MV) / ADC_MAX_VALUE;
}
```

References

[1] STMicroelectronics. RM0351 Reference Manual: STM32L47xxx, STM32L48xxx, STM32L49xxx, and STM32L4Axxx advanced Arm-based 32-bit MCUs. ADC, GPIO, and RCC register descriptions. https://www.st.com/resource/en/reference_manual/rm0351-stm32l47xxx-stm32l48xxx-stm32l49xxx-and-stm32l4axxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

[2] STMicroelectronics. STM32L476xx Datasheet, DS10198. Electrical characteristics, supply range, pinout, and ADC-related device information. https://www.st.com/resource/en/datasheet/stm32l476je.pdf

[3] STMicroelectronics. UM1724 User Manual: STM32 Nucleo-64 boards (MB1136). NUCLEO-L476RG connector mapping and Arduino A0 / PA0 information. https://www.st.com/resource/en/user_manual/um1724-stm32-nucleo64-boards-mb1136-stmicroelectronics.pdf

[4] ControllersTech. STM32 ADC Single Channel Polling Mode. https://controllerstech.com/stm32-adc-single-channel-polling/

[5] Digi-Key Maker.io. Getting Started with STM32: Working with ADC and DMA. https://www.digikey.com/en/maker/projects/getting-started-with-stm32-working-with-adc-and-dma/f5009db3a3ed4370acaf545a3370c30c